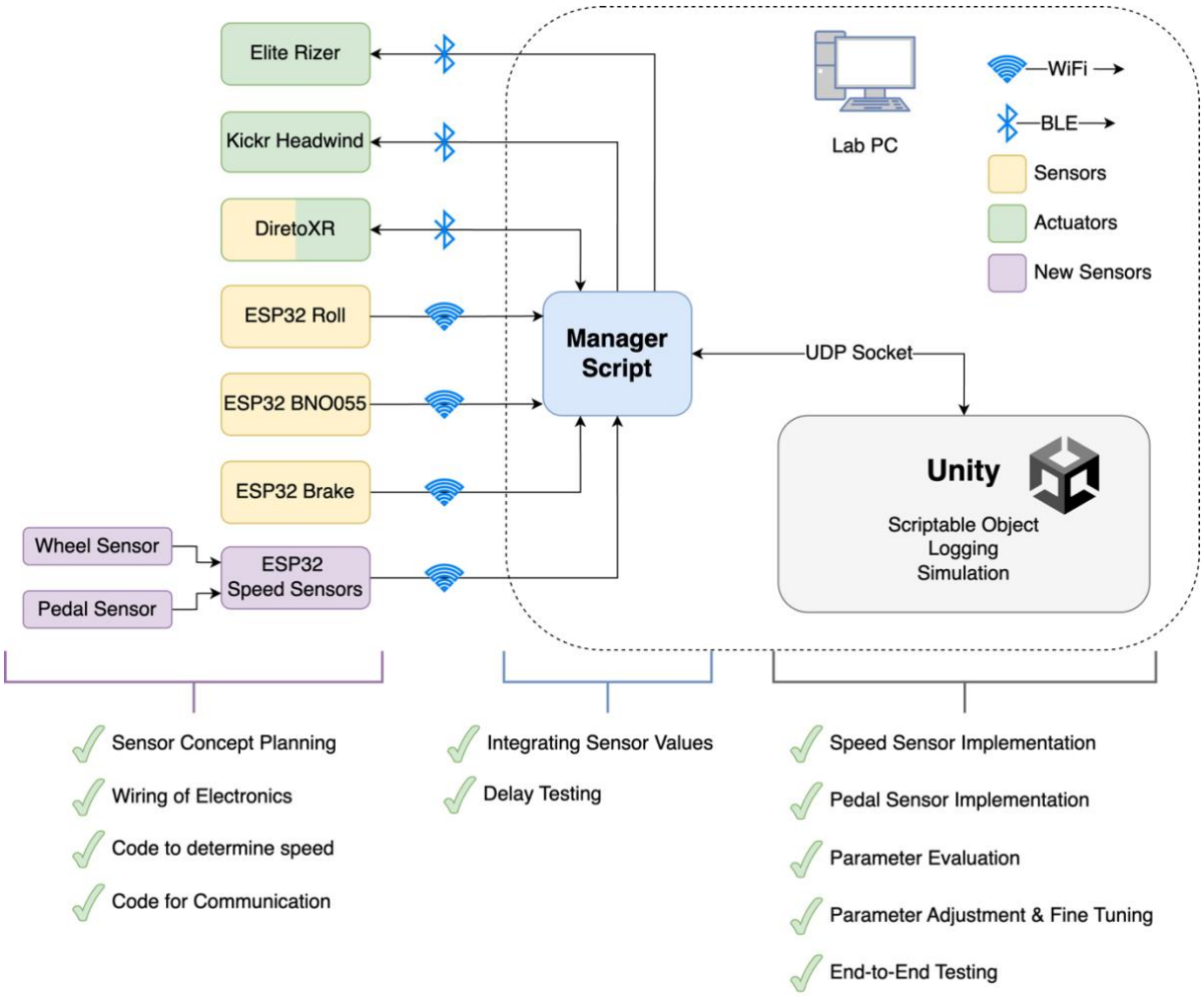
Documentation RTS Project

Adding Hall-Effect Sensors to Track Speed for Bicycle Simulation



Abstract

Cycling is an eco-friendly and efficient urban transportation mode, but safety concerns deter many potential users. To address these issues, Hochschule Heilbronn initiated the development of a bicycle simulator in 2022. This project focuses on enhancing acceleration measurement with the help of new sensors, optimizing resistance control, and improving the simulation's realism.

Key advancements include the introduction of a Back Wheel Sensor and a Pedaling Sensor. The Back Wheel Sensor tracks wheel rotation to calculate speed, while the Pedaling Sensor captures pedaling intensity for precise acceleration. Together, they deliver realistic cycling dynamics. To overcome delays from the Direto XR Trainer, two Hall Effect Sensor integrated with an ESP32 microcontroller were implemented, reducing sampling time to less than 100ms and greatly enhancing responsiveness.

The system is mounted on a rocker plate for lateral movement and integrated with a gaming PC and HTC Vive Pro VR glasses to create an immersive virtual reality environment. These improvements significantly enhance the realism and responsiveness of the cycling simulation, providing a strong foundation for future development.

Abbreviations

BLE	Bluetooth Low Energy
SoC	System on Chip
UDP	User Datagram Protocol
ISR	Interrupt Service Routine
VR	Virtual Reality

Table of Contents

Abstract **2**
Abbreviations **2**
1 Introduction **4**
2 Project Motivation and Goals **4**
3 Project Plan **5**
3.1 Orientation *5*
3.2 Execution *5*
3.3 Finalization *7*
4 Physical Bicycle Setup **8**
4.1 Commercial Devices *8*
4.2 Custom-Built Devices *9*
4.3 Integration and Communication *10*
4.4 Project Scope *10*
5 Speed Sensor **10**
5.1 General Concept *10*
5.2 Hardware Setup *10*
5.3 Software *11*
5.3.1 Version 1 12
5.3.2 Version 2 14
5.4 Measuring Perfomance Gains *16*
5.4.1 Hall-Effect Sensor Version 1 16
5.4.2 Hall-Effect Sensor Version 2 17
5.4.3 Sensor Comparisons 18
6 Pedal Sensor **18**
7 External Development Support **19**
8 Adjustments in Unity Simulation **19**
8.1 Simple Bicycle Physics Parameter Evaluation *19*
8.2 Resistance Calculation *19*
8.3 Pedal Simulation *20*
8.4 Addressing Acceleration Logic Issues *20*
9 Resolving Bluetooth Connectivity Issue **21**
10 Code Cleanup **21**
11 Further Improvements **21**
11.1 Adjusting Slowdown of the Sensor *21*
11.2 Power Supply *22*
11.3 Sensor Mounting *22*
12 References **23**

1 Introduction

The bicycle is a quick and eco-friendly mode of transportation in urban areas. Despite its advantages, many cyclists are deterred by safety concerns, as highlighted by public opinion studies on traffic infrastructure and road safety [1]. Bicycle simulation emerges as a promising method for addressing these concerns and enhancing the cycling experience. For that purpose, Hochschule Heilbronn started developing a bicycle simulator in 2022. This project continues the work of previous groups.

2 Project Motivation and Goals

This project focuses on enhancing the accuracy and responsiveness of acceleration measurement and the resistance actuator within the bicycle simulator. Several issues were identified at the outset that hindered the simulator's performance. One of the primary concerns was a significant **data transmission delay**, where acceleration data from the Direto XR Trainer experienced a noticeable lag before reaching the Unity simulation. This delay disrupted the fluidity and realism of the cycling experience.

Additionally, the **physics parameters** for the bicycle in Unity failed to accurately mimic real-world dynamics, resulting in unrealistic simulation behavior. Compounding these issues was an overly **complex and inefficient codebase** governing the bicycle's behavior. Originating from an external library, the script spanned 1,300 lines, of which approximately 70% were redundant and lacked relevance to the simulator's requirements. These inefficiencies emphasized the need for streamlined code and a more realistic approach to bicycle physics.

In addition to these challenges, specific behavioral issues further complicated the simulation's realism. Firstly, the brakes failed to function correctly when the bicycle was rolling backward, undermining the system's responsiveness in reverse scenarios. Secondly, the back wheel exhibited an unintended behavior where it continued spinning even after the bicycle had stopped. This residual motion caused the bike to lurch forward abruptly when the brakes were released, creating a disjointed and unrealistic user experience.

To address these issues and to create a more realistic and responsive cycling simulation environment, the following objectives were defined:

1. **Measure Data Transmission Delays**

Assess the delay in data transfer from the Direto XR Trainer to determine whether new sensors are required.

2. **Enhance Speed Transmission**

Attach sensors to the back wheel to improve the accuracy of speed measurement.

3. **Improve Acceleration Measurement**

Install sensors on the pedals to provide more precise acceleration data.

4. **Optimize Bicycle Physics in Unity**

Adjust Unity's bicycle physics parameters to ensure they more closely resemble real-world dynamics.

5. **Streamline the Codebase**

Refactor the existing code to improve efficiency and readability, removing unnecessary portions while retaining and enhancing critical functionality.

6. **Fix Behavioral Issues**

Resolve the issue with brakes not working during backward rolling.
Correct the problem of the back wheel spinning after the bike has stopped, ensuring consistent braking behavior.

3 Project Plan

3.1 Orientation

There were two possible main topics for the project, namely steering and acceleration. While improving the steering mechanisms to account for tilting of the rocker plate is a sensible enhancement, it is already planned to be addressed in a master thesis by another fellow student. As a result, the decision was made early on to concentrate on improving the acceleration functionality during this semester.

There were major flaws with the acceleration of the bicycle simulator as described in chapter “Project Motivation and Goals”. While it was clear that these flaws will be addressed, the specific goals and strategies to achieve this improvement were yet to be discovered in the initial stages of the conceptual process.

One goal that was sketched out and then discarded was the feature “Gear Tracking” which was initially considered but ultimately discarded during the conceptualization phase. This feature aimed to calculate a gear ratio based on data from the pedal and back wheel sensors. While the sensors themselves turned out to be extremely useful, the feature of Gear Tracking hardly adds any value to the simulation. The values provided by the back wheel rotation suffice for an accurate representation of the bicycle’s acceleration.

3.2 Execution

The implementation involves four layers in total: Hardware, Microcontroller, Manager Script and Unity Simulation. These will be explained in the following paragraphs.

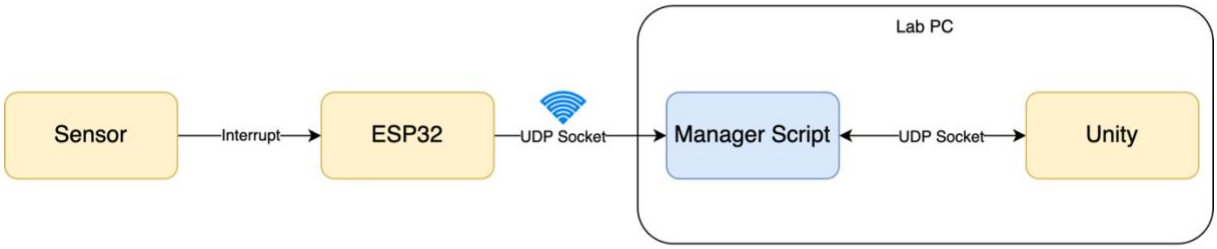


Figure 1: Dataflow between Sensor and Unity Simulation

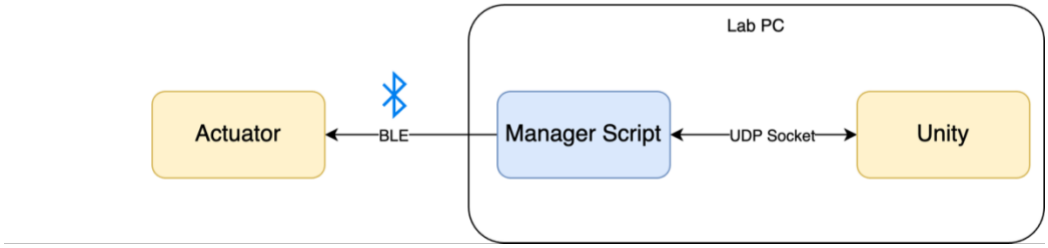


Figure 2: Dataflow between Unity Simulation and BLE Actuator

The **Hardware** layer encompasses the magnets, the Hall effect sensors, the housing for the microcontroller, and any cables and chargers attached. After assessing the requirements for the sensors, suitable alternatives within a reasonable price range were identified. The sensors were positioned on a part of the bicycle where the magnets would pass closely by, while ensuring the sensor itself was fixed in a specific position. The 3D housing for the microcontroller was fabricated by a fellow student, as detailed in the chapter "External Development Support."

We chose the ESP32-DevKitC 32E, which uses the ESP32-WROOM-32E Module Board, that has an integrated BLE and Wi-Fi Radio, giving us full flexibility for the Communication. The terms **Microcontroller**, Devkit and ESP32 are used interchangeably in this document, however they all refer to the ESP32 Devkit. Apart from an easy implementation of communication, the Devkit provides us with a Micro-USB Port, which can be used for Powering and Debugging, as well as a Pin, that can power it using a 5V Power Supply.

The **Manager Script** is a Python script running on the lab PC. It is the interface between Sensor and Actuators, that are connected to the PC via Bluetooth Low Energy (BLE) or Wi-Fi and the Unity Simulation, to which it connects using an on-device UDP-Socket connection. This abstraction makes it possible to integrate devices using different Protocols and Physical Interfaces. All proprietary and custom-built devices are described in the chapter "Physical Bicycle Setup."

The microcontrollers connected to the custom-built devices communicate with the Manager Script over the User Datagram Protocol (UDP) via Wi-fi. This allows for low-latency communication and is a standard that’s already implemented in the Manager Script. We enhanced the script to gather the data from our new sensor and forward it to the Unity Simulation.

The **Unity Simulation** integrates all the values provided by the manager script. To create an immersive experience, the data must be incorporated in a meaningful way. The possibilities for fine-tuning the interaction between the sensor values and the simulation are virtually limitless. While we succeeded in significantly improving the simulation with the new sensor data, we decided to focus on aspects of the project that fall within the scope of the subject "Real-Time Systems."

3.3 Finalization

While all the tasks related to the setup and integration of the new sensors can be attributed to one of the layers described above, the finalization of the project requires analysis and fine-tuning across all four layers simultaneously. To get a sensor value from the sensor into the simulation, several lines of code across all four layers must align and function cohesively. Once this alignment is achieved, potential improvements are identified, and the corresponding changes are assigned to a specific layer for implementation.

Throughout the project, we maintained regular communication and conducted multiple collaborative development sessions in the lab. As a result, the process of integrating the various components and ensuring they worked together was surprisingly smooth.

4 Physical Bicycle Setup

The bicycle simulator is designed to offer an immersive and realistic cycling experience by integrating both commercial and custom-built hardware components.

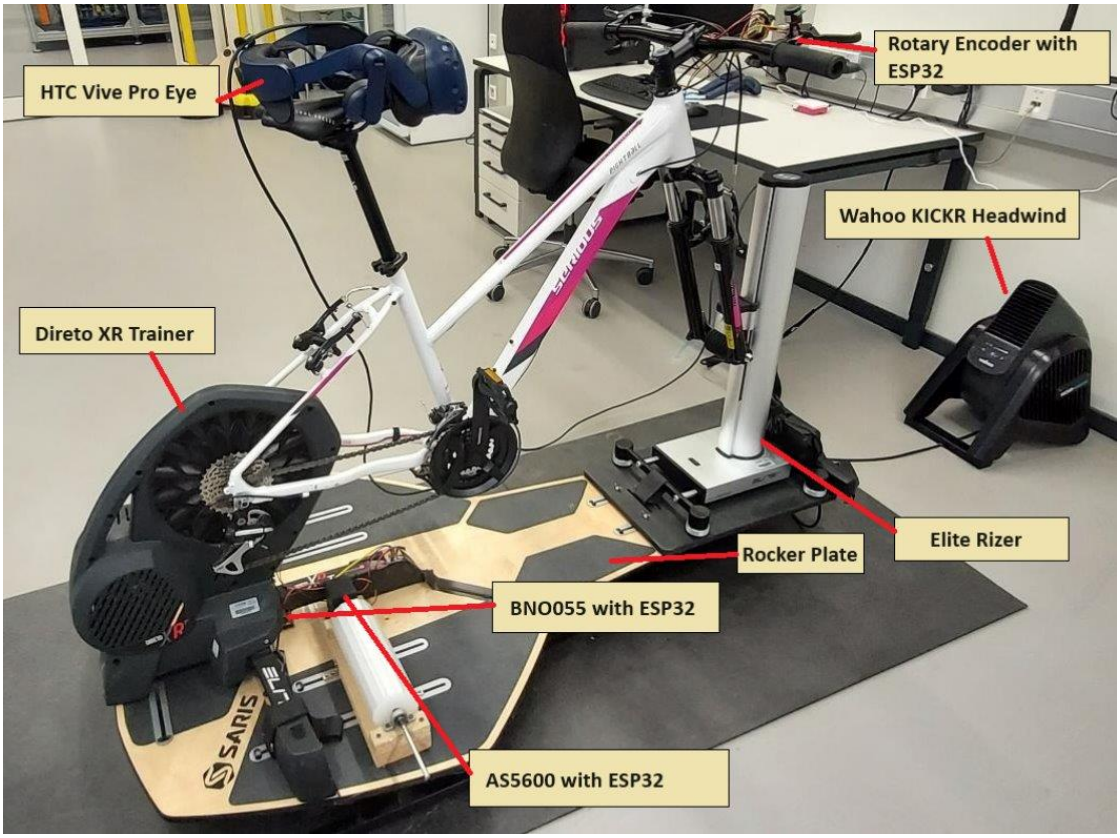


Figure 3: Physical Bicycle Setup

This section provides an overview of the physical setup of the simulator, including the devices used, their placements, and their functions.

4.1 Commercial Devices

Find the details of the integrated commercial devices listed below. All three are connected via the Bluetooth Low Energy (BLE) protocol.

The **Direto XR** is a state-of-the-art smart trainer positioned on the back axle of the bicycle, playing a critical role in the simulation. Its primary functionality is to provide real-time feedback on the cyclist's speed while enabling dynamic control of resistance. This capability allows the simulation to replicate varying terrains and inclines, offering a more realistic cycling experience. The device interacts with the system by reading the speed generated by the cyclist and receiving instructions to adjust resistance based on simulated conditions.

The **Elite RIZER** complements the Direto XR by focusing on the front axle to simulate changes in incline. It adjusts the bicycle's angle to mimic uphill and downhill scenarios, providing an additional layer of realism to the simulation. Through data interaction, the Elite RIZER receives

incline values from the system and seamlessly translates them into physical adjustments, enhancing the immersive quality of the ride.

The **KICKR Headwind** further elevates the cycling experience by simulating wind resistance. This device varies airflow based on the cyclist's speed, creating a dynamic and engaging environment. By interacting with the simulation, it receives headwind values calculated in real-time, ensuring the airflow matches the rider’s performance. Together, these devices work harmoniously to create a highly realistic and interactive cycling simulation.

4.2 Custom-Built Devices

Find the descriptions of the custom-built devices listed below. For all devices, data is sent over UDP to the main computer via the ESP32 SoC attached to the sensor which has built-in Wi-Fi.

The **Brake System** uses a rotary encoder to simulate the effect of braking, allowing the cyclist to bring the bike to a stop realistically. This system integrates seamlessly into the simulation to provide a tactile and responsive braking experience, enhancing the authenticity of the virtual ride.

The **Reverse Drive Mechanism** employs an AS5600 Hall Effect sensor to assist cyclists in reversing when stuck, mimicking the real-world action of pushing off from a stop. Data from the sensor is transmitted to the main computer over UDP using the ESP32's built-in Wi-Fi, ensuring smooth and accurate integration into the simulation.

The **Lean Sensor**, equipped with a BNO055 IMU, detects the cyclist's body lean during turns and adjusts the bike's rotation around the Z-axis within the simulation. This functionality captures the subtleties of leaning dynamics, adding depth and realism to the riding experience.

The **Steering Sensor** utilizes a photoelectrical encoder to track handlebar movement, translating it into adjustments in the bike's rotation around the Y-axis in the simulation. This sensor ensures that steering inputs are accurately reflected, enhancing the control and interactivity of the simulator.

The **Back Wheel Sensor**, a new addition, consists of magnets attached to the back wheel and a Hall Effect sensor. It monitors the wheel's rotation and measures its movement, sending this data to the simulation. The system uses this information to dynamically adjust the bicycle’s acceleration, creating a realistic representation of speed and momentum.

Another recent enhancement is the **Pedaling Sensor**, which features magnets affixed to the pedals and a Hall Effect sensor. This sensor tracks the pedals' rotation, capturing the frequency and intensity of pedaling. The collected data informs the simulation, ensuring that

acceleration corresponds accurately to the rider's effort, further improving the simulation's realism.

4.3 Integration and Communication

All devices, except for the KICKR Headwind, are mounted on a rocker plate, which allows for realistic lateral movement and balance adjustments during the ride. The system's architecture also includes a gaming PC equipped with HTC Vive Pro VR glasses, providing an immersive virtual reality environment.

4.4 Project Scope

At the outset of the project, the foundational functionality of all devices had already been implemented by previous teams. For the DiretoXR Trainer, speed-reading and resistance-writing capabilities were in place, but the communication between the trainer and the PC suffered from significant delays. Our primary objective was to improve the transmission of speed data to enhance the simulation’s responsiveness, particularly when starting the bike from a stationary position. This improvement directly contributes to the user's sense of immersion.

As outlined in “Project Motivation and Goals,” we addressed this issue by integrating input from both the back wheel sensors and the pedaling sensors. These two components are indispensable for creating a realistic cycling experience because they complement each other. The back wheel sensor captures the motion of the wheel, converting it into speed within the simulation. However, it cannot account for situations where the wheel keeps spinning due to inertia, such as when pedaling stops. To address this gap, the pedaling sensors track rider input directly, ensuring accurate simulation of acceleration and deceleration based on pedaling effort. By combining data from both sensors, we established a cohesive and responsive system that merges physical wheel movement with pedaling dynamics, significantly improving the realism of the cycling simulation.

5 Speed Sensor

5.1 General Concept

The speed sensor on the DiretoXR Trainer currently samples speed only once per second and experiences additional delay due to the Bluetooth Low Energy (BLE) protocol used for communication with the PC. To address this, we aimed to enhance the sampling rate and reduce transmission delays by implementing a Hall-Effect sensor in combination with an ESP32.

5.2 Hardware Setup

We mounted five magnets on the bicycle's back wheel and used a Hall-Effect sensor to measure the speed at which the magnets pass by. The sensor is digital, meaning it pulls the

voltage low when a magnet is nearby and outputs high voltage in its normal state. This behavior allows us to utilize an interrupt service routine (ISR) that is triggered whenever the signal transitions to low (at the falling edge). The sensor is connected to an ESP32, which communicates with the Lab PC using the User Datagram Protocol (UDP) over Wi-Fi.

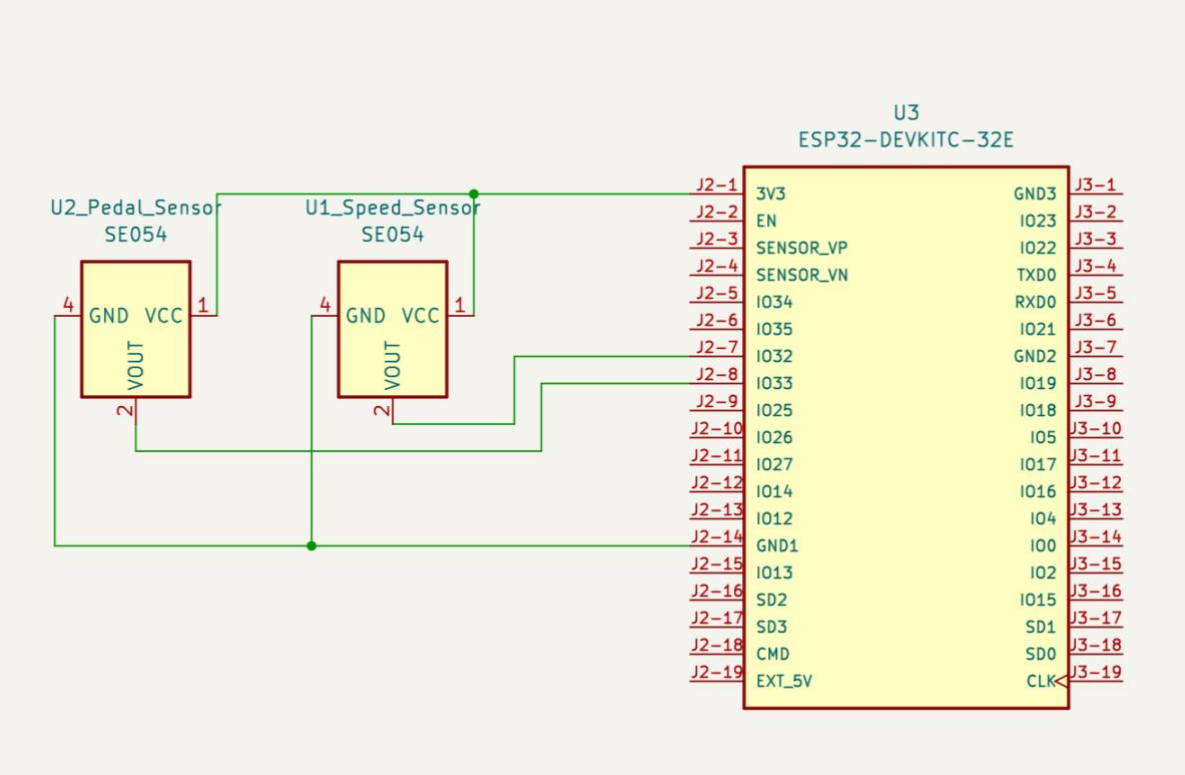


Figure 4: Wiring of Speed Sensor and Pedal Sensor using an ESP32-Devkit C – 32E

The wiring of the Sensor to the ESP32 is displayed in Figure 4. The two Hall-Effect Sensors are connected to a common Ground and 3.3V Rail. Each of them is connected to an IO-Pin that is capable of triggering an ISR.
The power was originally intended to be supplied through the 5V Pin, however because of problems with the power distribution, that are described in Chapter 11, we powered the ESP through its Micro USB Port.

5.3 Software

Version 1 of the Hall-Effect sensor was implemented using a sliding average window with a duration of 1 second. It counts the number of ticks using an interrupt service routine and executes the main function cyclically every 20 ms. In contrast, Version 2 of the Hall-Effect sensor measures the time difference between interrupts, making it independent of the main function's timing accuracy. The key advantages and drawbacks of each sensor version will be discussed in the following chapter.

5.3.1 Version 1

The interrupt function is used to increment the variable “count” every time that a magnet passes by. This variable is then pushed to an array of size 50 in the main function every 20ms, making the array hold the number of counts for the past second. The speed can now simply be calculated from the first and last element of the array. Find the flowchart and code in the images below.

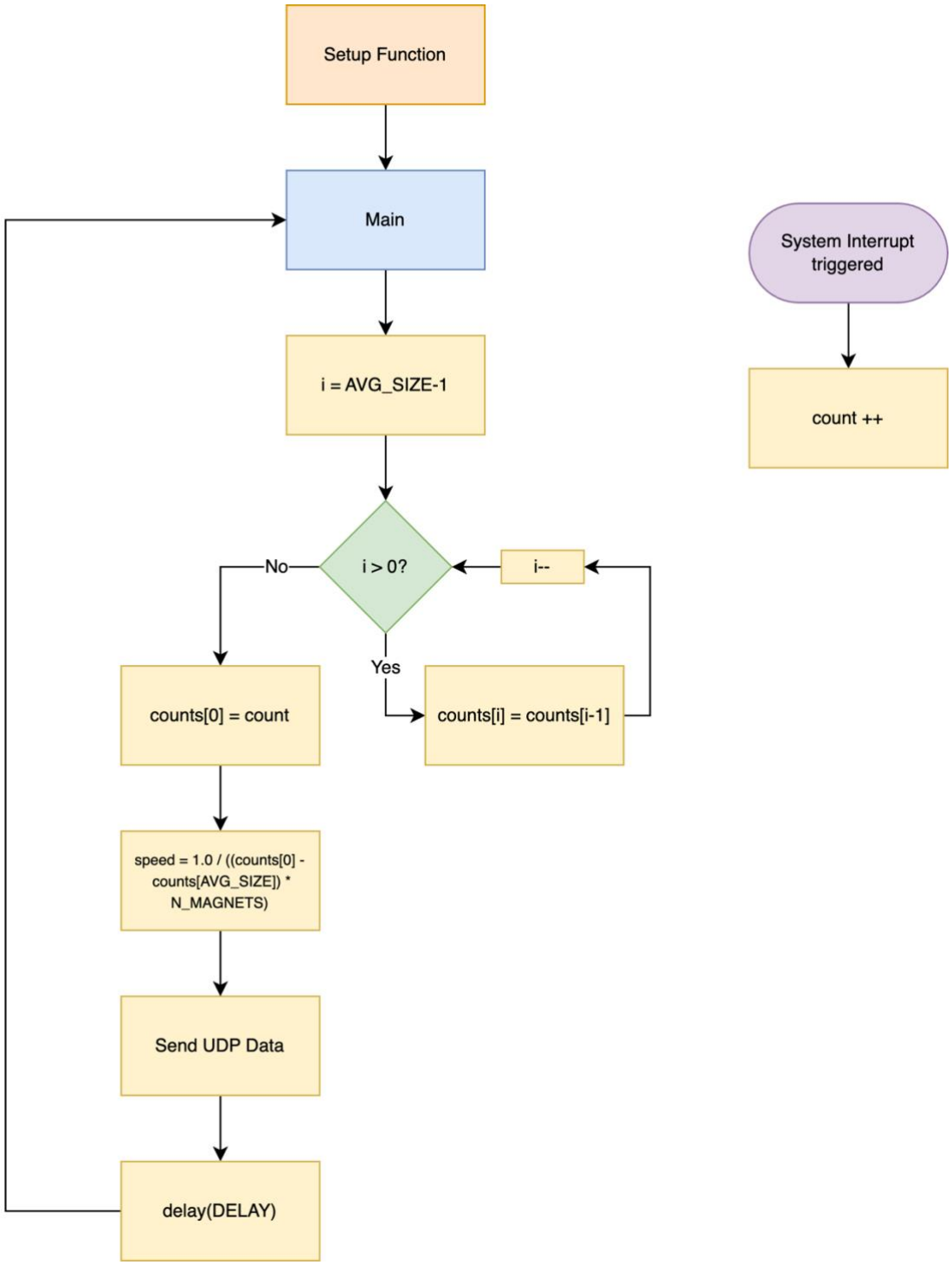


Figure 5: Flowchart of Hall-Effect Sensor Version 1

```
// WIFI
...

// SENSOR
const int HALL_PIN = 32;       // Hall-Effect Sensor Pin
const int UDP_DELAY_MS = 20;   // Delay in main
const int N_MAGNETS = 5;       // Number of Magnets
const int AVG_SIZE = 50;       // Sliding average window size
const int AVG_DELAY = UDP_DELAY_MS * AVG_SIZE;

// metrics for speed calculation
struct rotationMetrics {
  int count = 0;
  int counts[AVG_SIZE] = {0};
  float speed = 0;
};
rotationMetrics metrics;


void IRAM_ATTR isr() {
  metrics.count++;
}

void setup() {
  pinMode(HALL_PIN, INPUT_PULLUP);
  attachInterrupt(HALL_PIN, isr, FALLING);

  // Setup for UDP, etc.
  ...
}

void loop() {
  // Calculate speed
  for (int i=AVG_SIZE-1; i>0; i--) {
    metrics.counts[i] = metrics.counts[i-1];
  }
  metrics.counts[0] = metrics.count;
  metrics.speed = (1000.0 * (metrics.counts[0] - metrics.counts[AVG_SIZE-1])) /
float(AVG_DELAY * N_MAGNETS);

  // Sending Data via UDP, etc.
  ...
}
```

Code 1: C++ Code Snippet of Sensor Version 1

As you can see from the code above, the calculation of the speed is done by only the first and last elements in the array “metrics.counts”. Once a loop is finished, all values in the array are pushed to the right and the current “metrics.count” is appended.
This implementation therefore represents a sliding average algorithm, where the current speed is always the average over the past second.

5.3.2 Version 2

In Version 2 of the Sensor, the interrupt is used to save the current time and the time-difference to the previous interrupt in the variables “previous_time” and “time_difference”. This time-difference can be directly used in the main function to calculate the current speed. However, if the wheel stops completely, no more interrupts are triggered, meaning the sensor would continuously output the last recorded speed. This can be overcome using an if statement, as can be seen in the flowchart and code below.

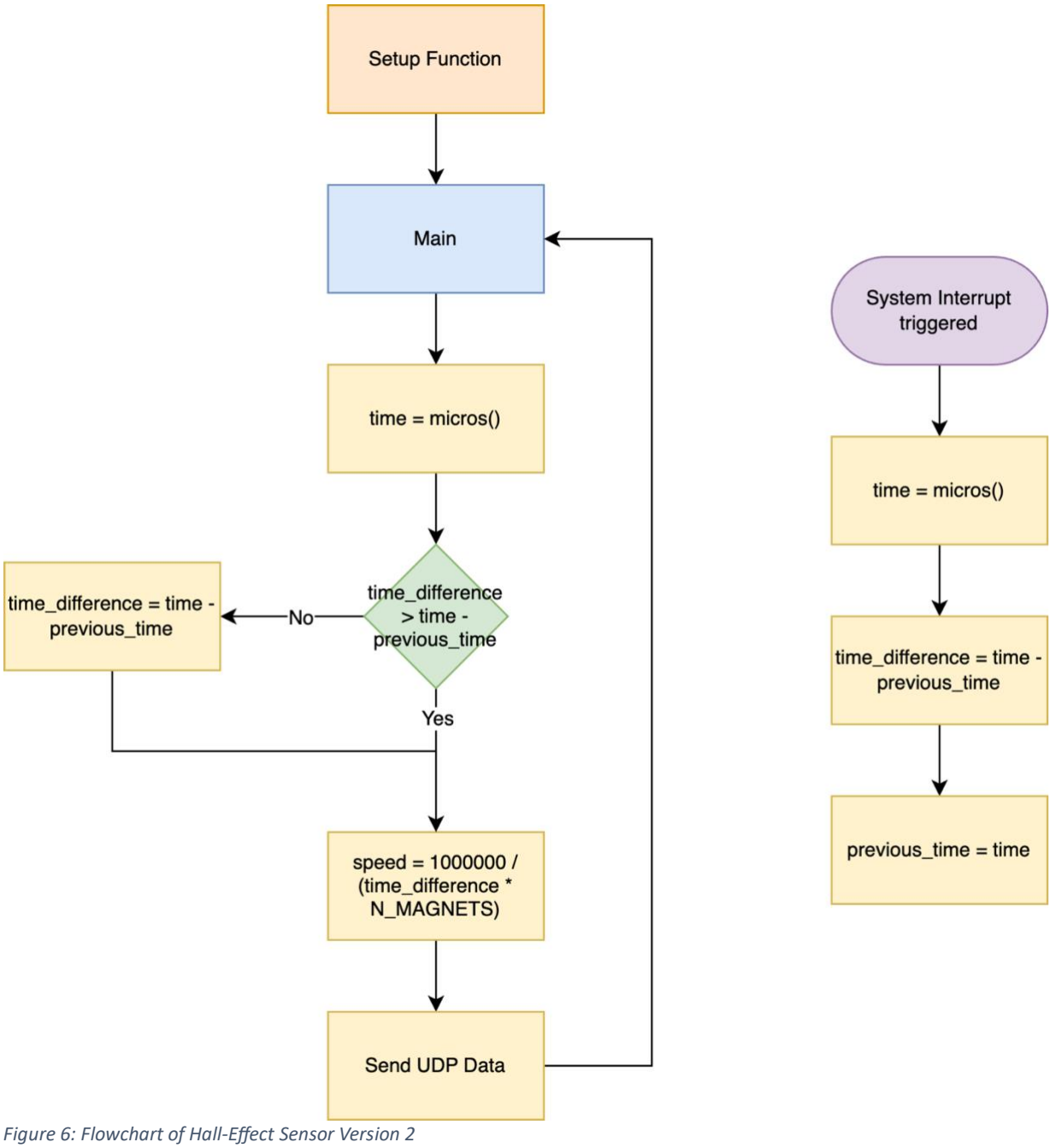


Figure 6: Flowchart of Hall-Effect Sensor Version 2

```
// WIFI
...

// SENSOR
const int HALL_PIN = 32;       // Hall-Effect Sensor Pin
const int UDP_DELAY_MS = 20;   // Delay in main
const int N_MAGNETS = 5;       // Number of Magnets

// metrics for speed calculation
struct rotationMetrics {
  uint64_t time_difference = 0;
  uint64_t time = 0;
};
rotationMetrics metrics;

void IRAM_ATTR isr() {
  int current_time = micros();
  metrics.time_difference = current_time - metrics.time;
  metrics.time = current_time;
}

void setup() {
  pinMode(HALL_PIN, INPUT_PULLUP);
  attachInterrupt(HALL_PIN, isr, FALLING);

  // Setup for UDP, etc.
  ...
}

float calculate_speed(rotationMetrics metrics, int n_magnets) {
  uint64_t time = micros();
  uint64_t time_difference = 0;

  // reduce speed gradually if no more interrupts are occuring
  if (time - metrics.time > metrics.time_difference) {
    time_difference = time - metrics.time;
  }
  else {
    time_difference = metrics.time_difference;
  }

  float speed = 1000000.0 / (time_difference * n_magnets);

  // clip low speeds
  if (speed < 0.01) {
    speed = 0;
  }

  return speed;
}

void loop() {
  // Calculate speed
  float speed = calculate_speed(metrics, N_MAGNETS);

  // Sending Data via UDP, etc.
  ...
}
```

Code 2: C++ Code Snippet of Sensor Version 2

Note: The actual code of Version 2 contains the speed calculation for both the backwheel as well as the pedal, using two interrupt service routines. For a better comparison against Version 1, we removed the pedal speed sensor and renamed the variables in the code above accordingly.

5.4 Measuring Perfomance Gains

To evaluate the performance of our sensor, we compared it to the Direto sensor by logging the measurement in the “master_collector.py” script. The Dataflow Images Figure 1 and Figure 2 illustrate that we disregarded the delay between the master_collector.py script and the Unity simulation. However, since both the Direto sensor and Hall-Effect sensor communicate to the simulation through the same data path, we can assume a similar delay.

5.4.1 Hall-Effect Sensor Version 1

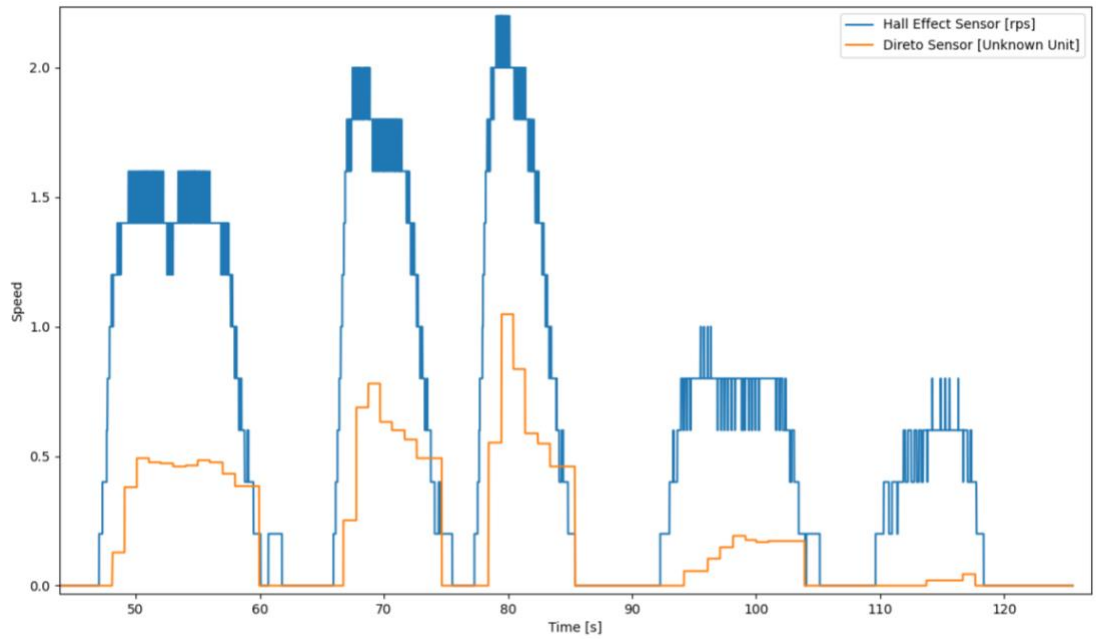


Figure 7: Delay Test using the Hall-Effect Sensor in Version 1 (Blue) and Direto Sensor (Orange)

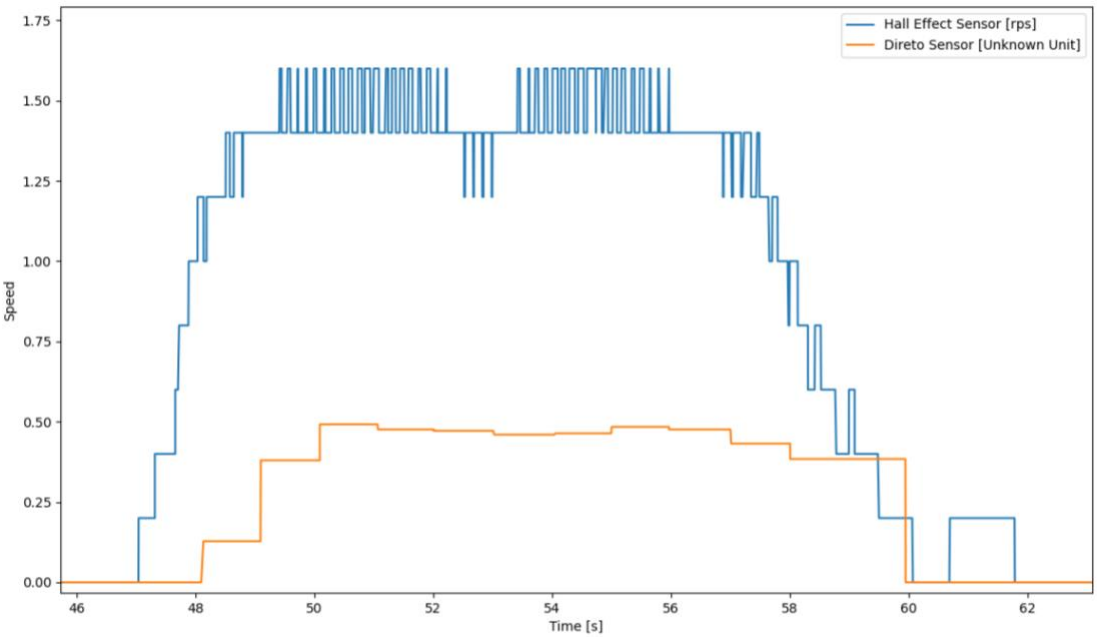


Figure 8: Zoom into Delay Test first speed up

Figure 7 and Figure 8 demonstrate that the Hall-Effect sensor responds approximately 1 second faster than the Direto sensor. The Hall-Effect sensor has a sampling time of less than

100 ms, compared to the Direto sensor's sampling time of 1 second. However, the Direto sensor provides significantly better speed resolution than the Hall-Effect sensor. This difference is due to the implementation of a 1-second sliding average window in the Hall-Effect sensor, which only allows a resolution of $v_{LSB} = \frac{1}{n_{Magnets}} = \frac{1}{5} rps$.

5.4.2 Hall-Effect Sensor Version 2

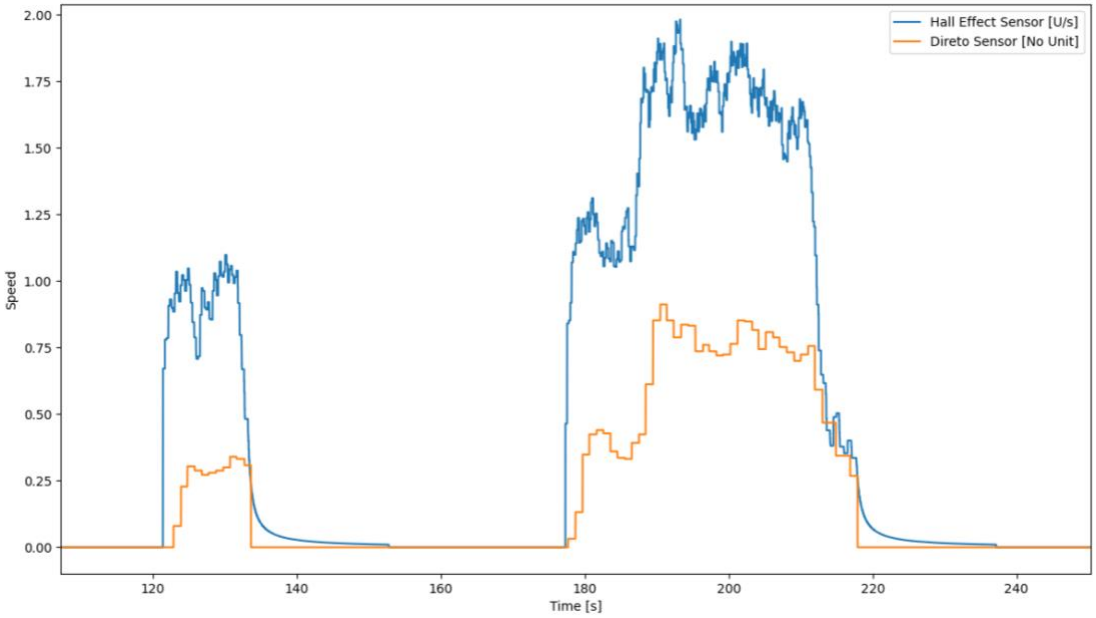


Figure 9: Delay Test using the Hall-Effect Sensor in Version 2 (Blue) and Direto Sensor (Orange)

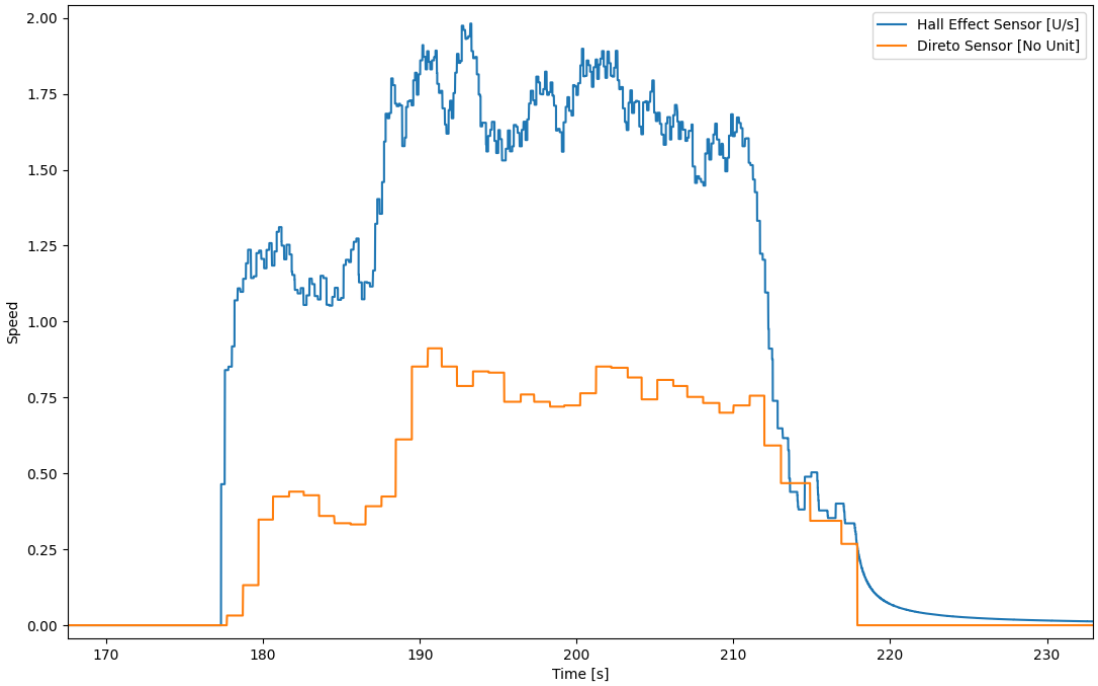


Figure 10: Zoom into Delay Test second speed up

Figure 9 and Figure 10 show that with Version 2 of the sensor, speed resolution is significantly improved. Additionally, low speeds are now reported accurately. Since no

averaging is applied, rapid increases and decreases in speed are also captured with high precision.

However, one issue with this implementation arises when the speed decreases to the point where no magnets pass by the sensor. Since the speed is calculated using the equation $v = \frac{1}{t_{Diff} \cdot n_{Magnets}}$, slowing down to a stop results in an exponential decay in the speed measurement, meaning it will approach 0, but never reach it. This behavior is evident at the very end of the graph in Figure 10.

From a physical standpoint, this behavior resembles a system with a certain half-life, which deviates from the actual physics of our system. In reality, resistance reduces rotational speed in a more linear manner. To address this issue, we opted to cap the speed at a minimum of 0.1 revolutions per second (rps). This threshold is sufficiently low for real-world scenarios and does not compromise the immersion.

5.4.3 Sensor Comparisons

During the initial testing of the simulator, we observed a delay from the Direto sensor of about 2.5s to 3s. During our studies, another student working simultaneously on BLE communication managed to reduce this delay to about 1s. Our Hall effect sensor reduced the delay even further. The results are shown in the table below.

Sensor	**Direto Original**	**Direto Improved**	**Hall-Effect Sensor V1**	**Hall-Effect Sensor V2**
Delay	~ 2.5s – 3s	~ 1s	~ 0.1s	~ 0.1s
Resolution	Unknown	Unknown	0.2 rps	“No Limit”
Minimum Speed	Unknown	Unknown	0.2 rps	0.1 rps

6 Pedal Sensor

The physical implementation of the pedal speed sensor is identical to that of the backwheel speed sensor, with the only difference being that we mounted 4 magnets instead of 5. In the software, we implemented an additional interrupt on Pin 33, which uses its own metrics structure. The data from the pedal speed sensor is then sent via the same UDP connection as the backwheel speed sensor.

The pedal sensor is used to accurately display pedal movement in the simulation. Additionally, it serves to improve the simulation in certain edge cases, such as when the backwheel continues rotating after coming to a complete stop in the simulation. These edge cases will be discussed in chapter 8.4.

7 External Development Support

A fellow student, who had previous experience with the bicycle simulator but was not directly involved in the current project, provided valuable support by 3D printing a sensor housing. This housing had already been successfully used twice for other sensors, demonstrating its reliability and utility.

In addition, the same student helped improve the delay of the BLE devices by adjusting the communication between the internal UDP socket connection and the BLE connection. He implemented two asynchronous tasks that communicate via a queue, which reduced the delay between Direto and the manager script to about 1s, optimizing the system's responsiveness and making the resistance actuator much more realistic.

8 Adjustments in Unity Simulation

This chapter describes how the values from the sensors described are integrated into the bicycle simulation.

8.1 Simple Bicycle Physics Parameter Evaluation

To improve the bicycle simulation, we conducted research on adjustable parameters to enhance realism and usability. One of the key adjustments involved the braking mechanism. The **threshold for brake activation** was moved farther from the default value on bicycle startup, starting with a difference of 40 instead of the previous 10. This change reduces brake sensitivity, preventing unintended activation caused by the natural fluctuations of a sensor that is easily triggered. Additionally, we introduced the **capability for braking while moving backward**, a feature that was previously absent.

We **increased the combined mass of the bicycle and rider** from 10 to 60, significantly enhancing the simulation's realism. Alongside this, we modified the acceleration curve to provide smoother acceleration, accommodating the increased mass. The **top speed was also raised** from 12 to 30 km/h, further improving the realism of the simulation.

Parameters related to steering, such as axisAngle and steerAngle, were also evaluated. While these showed potential for enhancement, no changes were implemented due to the complexity of steering dynamics. This topic will be addressed in greater detail in a master's thesis by another student.

8.2 Resistance Calculation

The total resistance calculation was updated to **include brake resistance and speed resistance** in addition to the incline resistance that was already being calculated. Brake resistance accounts for manual braking input, significantly slowing down the back wheel during the braking process. Speed resistance models the impact of wind resistance at higher speeds. To

ensure proper functioning, a clamping mechanism is used to keep the total resistance within the range of 0 to 256, which is the range supported by the DiretoXR device.

Find the code for the key part of this calculation below. The total resistance calculation incorporates brake resistance, speed resistance, and incline resistance. The combined incline and speed resistance is capped at a maximum value of 100 to ensure that normal cycling at high speeds or on steep inclines remains feasible. However, brake resistance can push the total resistance above 100, reaching up to 256, as braking forces should prevent the cyclist from accelerating.

```
// Calculate additional resistance based on speed
float speedFactor = 0.3f;
float speedResistance = Mathf.Round(speed * speedFactor);

// Total resistance is the sum of incline and speed resistance
float intermediateResistance = Mathf.Clamp(inclineResistance + speedResistance, 0, 100);

// Calculate additional resistance based on the brake input
float brakeResistance = Mathf.Round(brake * 400f);

float totalResistance = Mathf.Clamp(Mathf.Round(intermediateResistance + brakeResistance), 0, 256);
```

Code 3: Resistance Calculation in the Unity Simulation Using Incline, Brake and Speed Values

8.3 Pedal Simulation

Previously, the pedals in the simulation did not move, limiting the realism of the experience. With the introduction of new pedal sensors, we are now able to control the pedal movement in a sensible and responsive manner. To refine this further, a threshold was added to the movement mechanism, ensuring the pedals remain stationary when the sensor values are minimal due to the flattening curve. This adjustment prevents unnecessary or unrealistic pedal movements, enhancing the overall quality of the simulation.

```
if (bikeData.pedalSpeed < 0.2f) {
    bikeData.pedalSpeed = 0;
}
crankSpeed += bikeData.pedalSpeed * 360 * Time.deltaTime;
crankSpeed %= 360;
cycleGeometry.crank.transform.localRotation = Quaternion.Euler(crankSpeed, 0, 0);
```

Code 4: Update Procedure of the Pedal Visuals in the Unity Simulation Using the Pedal Sensor Values

8.4 Addressing Acceleration Logic Issues

A significant issue was identified in the simulation where, after braking, the bicycle would accelerate on its own once the brakes were released. This unintended acceleration occurred due to the force generated by the spinning back wheel, even though no pedaling input was present. To resolve this, we adjusted the system to apply acceleration only when values are received from the pedal sensors, ensuring realistic behavior. Furthermore, to mitigate the back wheel's rotation during braking, we introduced high resistance on the Direto, the device simulating the back wheel, effectively slowing it down and preventing unwanted motion. These changes improved the accuracy and realism of the simulation.

```
if (pedalSpeed > 0.2f) {
    rawCustomAccelerationAxis = backwheelSpeed * 0.5f;
}
```

Code 5: Acceleration Logic in the Unity Simulation Based on the New Sensors

9 Resolving Bluetooth Connectivity Issue

A connectivity issue arose when attempting to use the bicycle simulator after a Windows update on the Lab PC. The Direto XR Trainer was unable to connect via Bluetooth, preventing the simulator from functioning properly. Initial debugging revealed that the BleakScanner package, responsible for detecting Bluetooth devices, was not finding any devices. To verify the problem, we tested connecting to the Direto using a mobile device, which successfully established a connection, confirming the issue was isolated to the Windows setup.

Upon further investigation, we discovered that the Windows update had turned off the Bluetooth functionality by default. Once this was identified, we re-enabled Bluetooth and ensured the operating system was fully updated, resolving the issue. This fix restored the simulator's connectivity and ensured proper operation of the Direto device.

Though this issue is described briefly, the analysis took considerable time, interrupted the development workflow, and may arise in future updates as well. Therefore, we decided to document it here to raise awareness for future research and development in this project.

10 Code Cleanup

A significant effort was dedicated to streamlining the existing codebase to enhance maintainability and performance. The "Old Version" toggle was removed, along with extensive sections of commented-out code that no longer served a purpose. Code for keyboard control, which was deemed irrelevant to the current sensor-based implementation, was also eliminated. Additionally, handling for mid-air behavior, which was unnecessary for the simulation context, was excluded. These adjustments resulted in reduced code complexity and improved readability, allowing for more efficient development and debugging processes.

11 Further Improvements

11.1 Adjusting Slowdown of the Sensor

The problem of the sensor not decreasing speed quickly enough can be addressed in several ways which are listed below.

1) An additional factor can be introduced to decrease the speed exponentially, counteracting the exponential decay described in Chapter 5.4.2. This adjustment can

be implemented in the calculate_speed function using the following code snippet. Note that the factors (e.g., 3 and 0.1) must be fine-tuned through testing.

```
  // reduce speed gradually if no more interrupts are occuring
  if (time - metrics.time > metrics.time_difference) {
    time_difference = time - metrics.time;
    time_difference = time_difference + pow(time_difference -
metrics.time_difference, 3) * 0.1
  }
  else {
    time_difference = metrics.time_difference;
  }
```

Code 6: C++ Code Snippet for adjusting slow speeds using exponential factors

This approach helps maintain realistic speed behavior by reducing speed appropriately when no magnets are detected, while still preserving the system's responsiveness.

2) Another approach involves storing the previous speed decreases by writing the time differences into an array and calculating a resistance factor. Using this factor, a linear decrease in speed can be estimated based on the historical decreases, ensuring smoother and more realistic deceleration.

3) The simplest approach is to add more magnets to the system. This would decrease the sampling time at slower speeds, significantly reducing the impact of the problem.

11.2 Power Supply

When testing the Hall-Effect Sensor, we noticed, that one of the three other ESP32s would shut down and reboot constantly. We investigated this Issue and found out, that the 5V Power Supply struggled to maintain a constant voltage over the 3m wires to the ESPs.
Due to the limited scope of this project, we temporarily fixed this, by supplying 3 of the 4 ESPs through the Micro-USB connectors and giving each their own Power Supply.

However, for Future Improvements, a new Power concept should be thought of. Since each of the ESPs has a Linear Dropout Regulator (LDO), they could be powered using a 12V power supply, reducing the voltage drop over the long wires.
Additionally, adding Capacitors to the 3.3V Rail of the ESP32s reduces peak current consumption and would help to keep the supply line at a stable voltage.

11.3 Sensor Mounting

During the project, we mainly focused on improving the realism of the simulation and testing our solutions. The mounting of the sensor was never permanent, using tape allowed us to easily mount the sensor in different positions.
However, we have designed a 3D printable mounting mechanism, as shown in Figure 11. It still needs to be printed and attached to the frame of the bike.

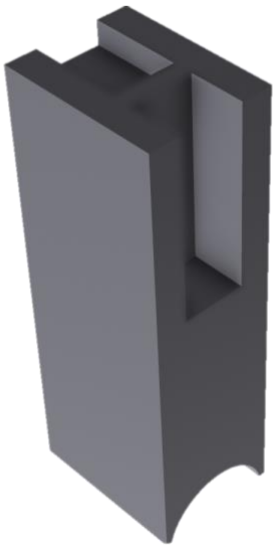


Figure 11: 3D printable mounting mechanism for the Hall-Effect sensor

12 References

Statista and YouGov. 2024. YouGov-Umfrage zum Straßenverkehr 2024. Statista and YouGov. https://de.statista.com/presse/p/yougov_umfrage_stra_enverkehr_2024/ Accessed: 2024-12-14.